



Introducción a las Aplicaciones Web

Universidad de Sevilla

Departamento de Lenguajes y Sistemas
Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Objetivos



Contenidos

Introducción a las aplicaciones web

Arquitecturas web

Servicios Web

Desarrollo front-end



Contenidos

Introducción a las aplicaciones web

Arquitecturas web

Servicios Web

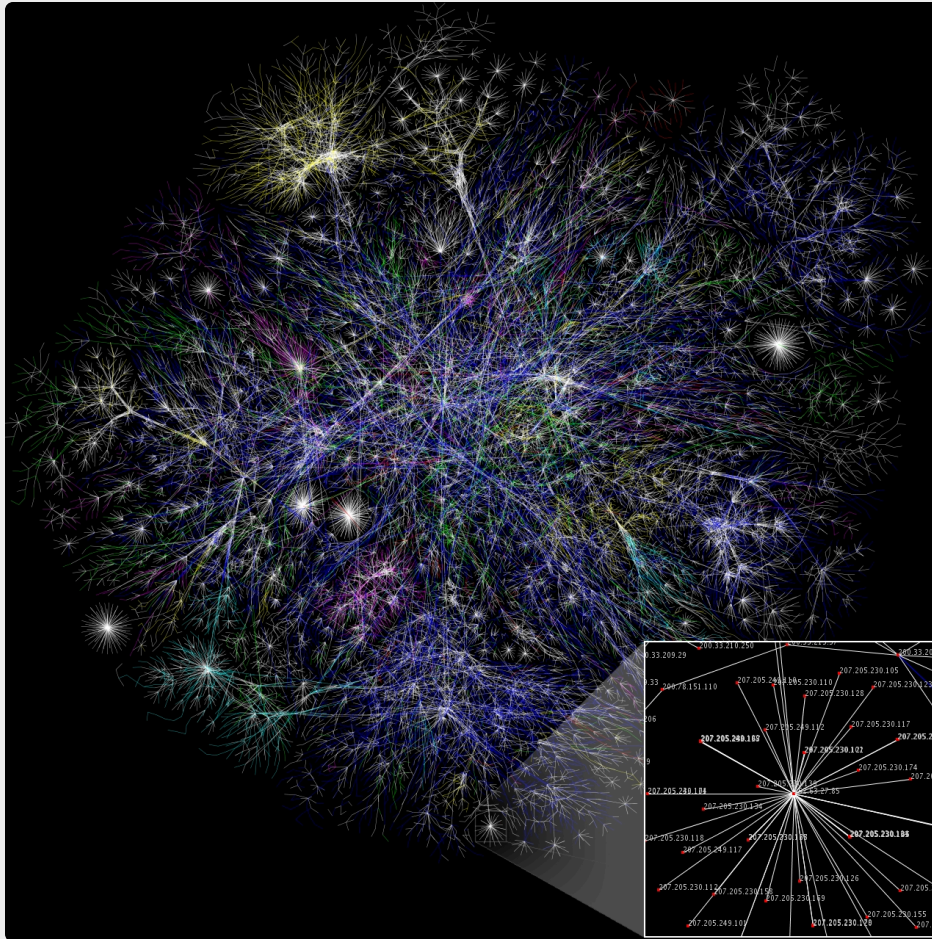
Desarrollo front-end



Internet

Conjunto descentralizado de redes de comunicación interconectadas que utilizan la familia de protocolos TCP/IP y forman una red lógica única de alcance mundial

(Fuente: [Wikipedia.org](https://es.wikipedia.org/wiki/El_Palacio_Nacional_de_Cuba))



Internet vs La Web

Internet

- “INTERconnected NETworks” o “INTERnational NET”
- Red de redes
- Infraestructura que da soporte a la web, entre otros servicios

La Web

- World Wide Web (www)
- Colección de datos (información) a la que se accede a través de Internet
- Servicio que se ofrece sobre Internet

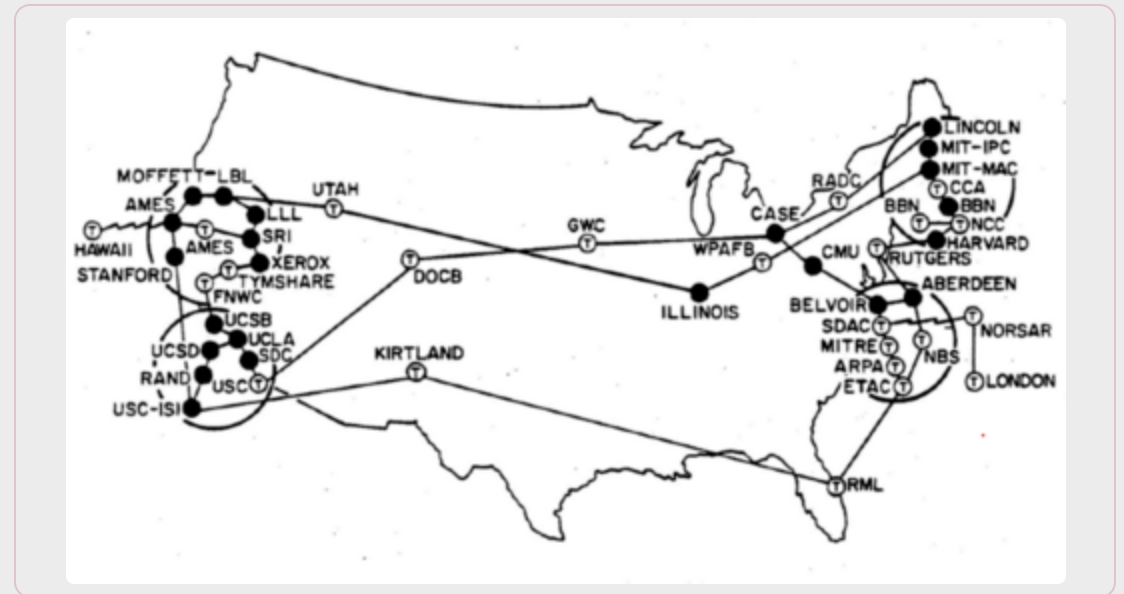


Internet

Hija de ARPANet, red de la DARPA (Defense Advanced Research Projects Agency, Agencia de Defensa de los EEUU).

- **Objetivo:** Comunicar los ordenadores de distintos laboratorios entre sí.

(Fuente: [Wikipedia.org](https://es.wikipedia.org/wiki/El_mito_de_la_Independencia))



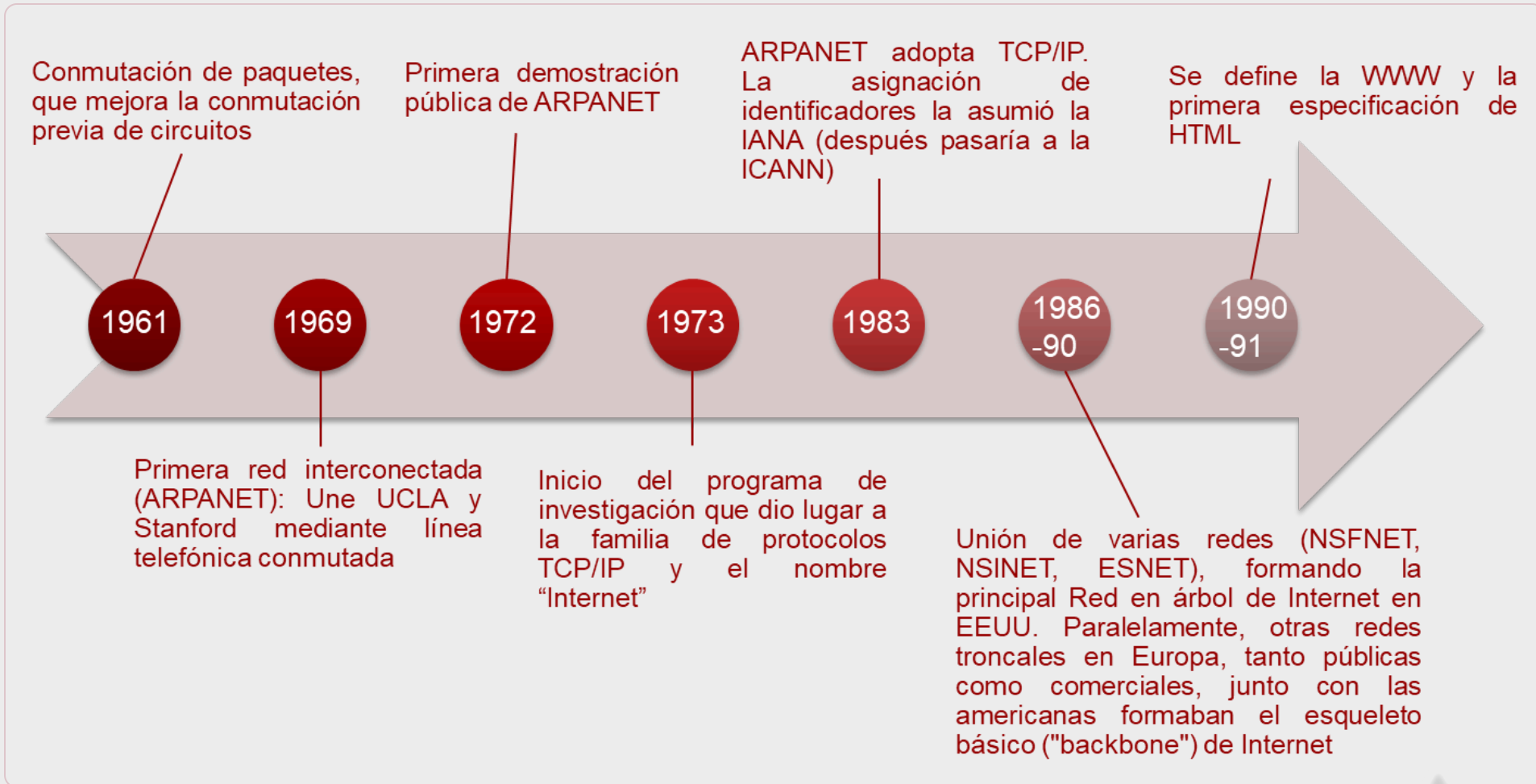
Entidades detrás de Internet

Internet no tiene una gobernanza centralizada única ni en la implementación tecnológica ni en las políticas de acceso y uso; cada red constituyente establece sus propias políticas.

Entidades involucradas en su gestión:

- **Grupo de Trabajo de Ingeniería de Internet (IETF)**, una organización internacional sin fines de lucro: base técnica y la estandarización de los protocolos centrales
- **Corporación de Internet para la Asignación de Nombres y Números (ICANN)**: corporación internacional sin ánimo de lucro: asignación de direcciones IP, identificadores de protocolo, gestión de los nombres de dominio y administración de servidores raíz. Coordina la administración de los elementos técnicos del Sistema de Nombres de Dominio (DNS) para garantizar la resolución unívoca de los nombres

Línea temporal



TCP/IP

Se basa en la encapsulación/desencapsulación de datos pasando por distintas capas

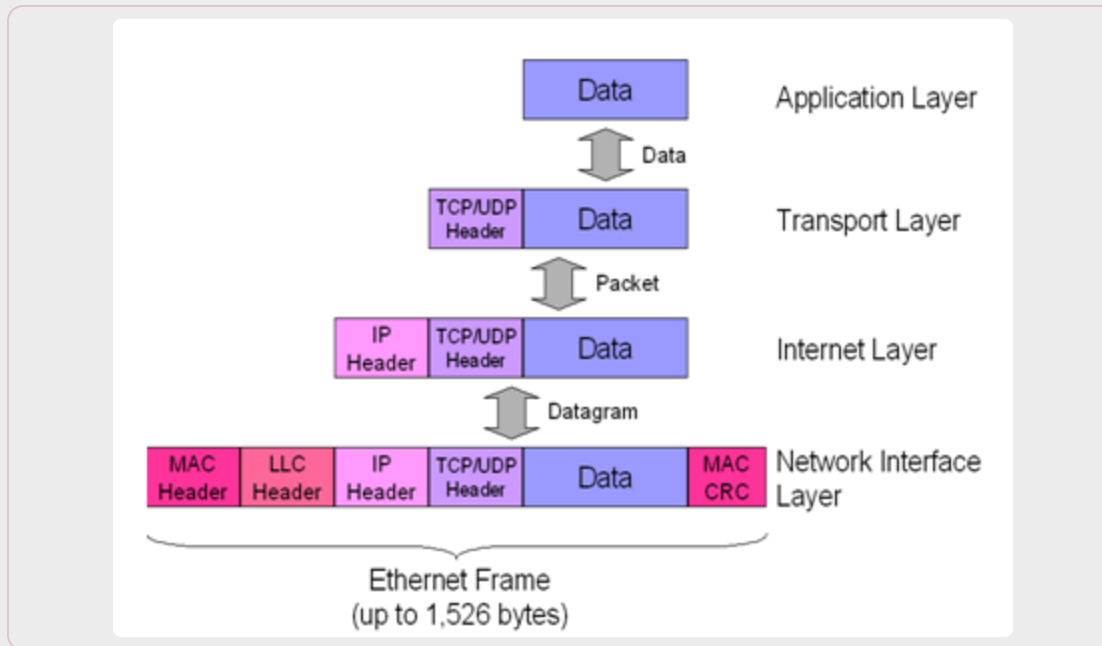
- Cada capa añade una cabecera (a veces también cola), con la información de control para la capa superior

La capa física se encarga de la transmisión a través de internet

La capa IP se encarga del direccionamiento hacia ordenadores concretos identificados por una dirección IP

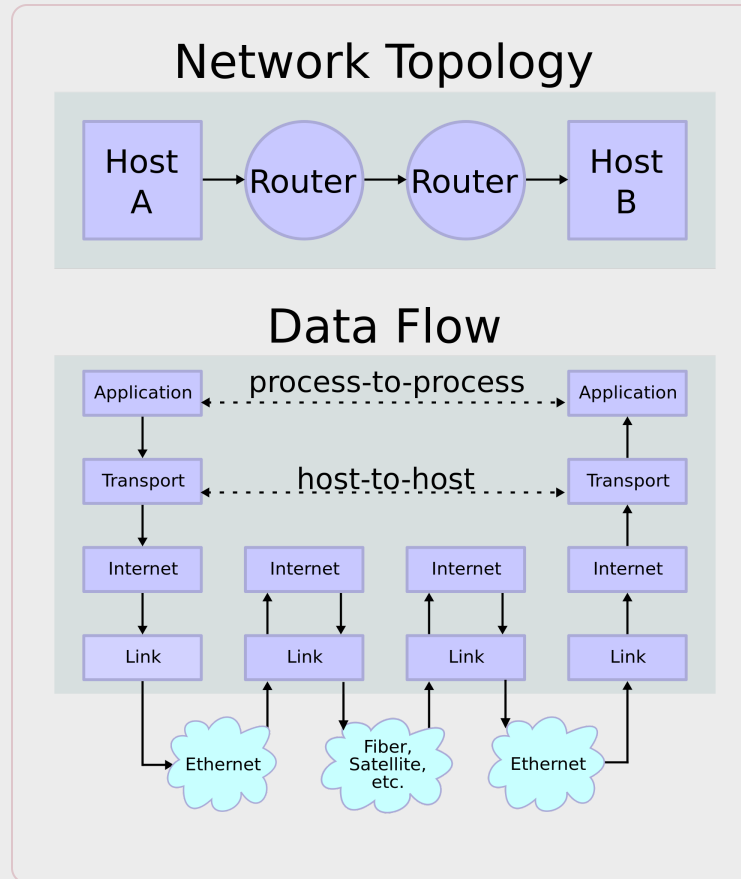
La capa (TCP) se encarga de seleccionar el proceso o aplicación que se está comunicando, mediante un número de Puerto

- Ejemplo: HTTP: 80, FTP: 20-21, SMTP: 25



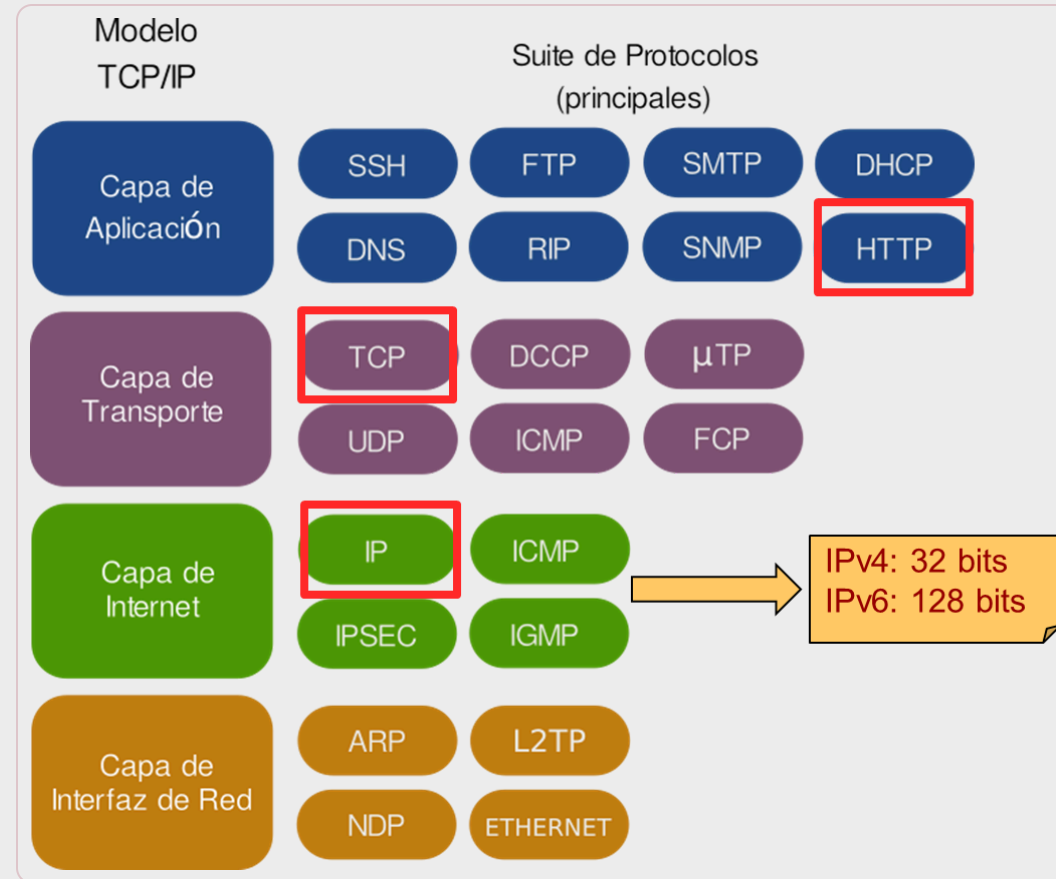
Fuente: <https://mudnaved.wordpress.com/tag/tcpip-suite/>

Flujo de comunicación



Fuente: Wikipedia.org

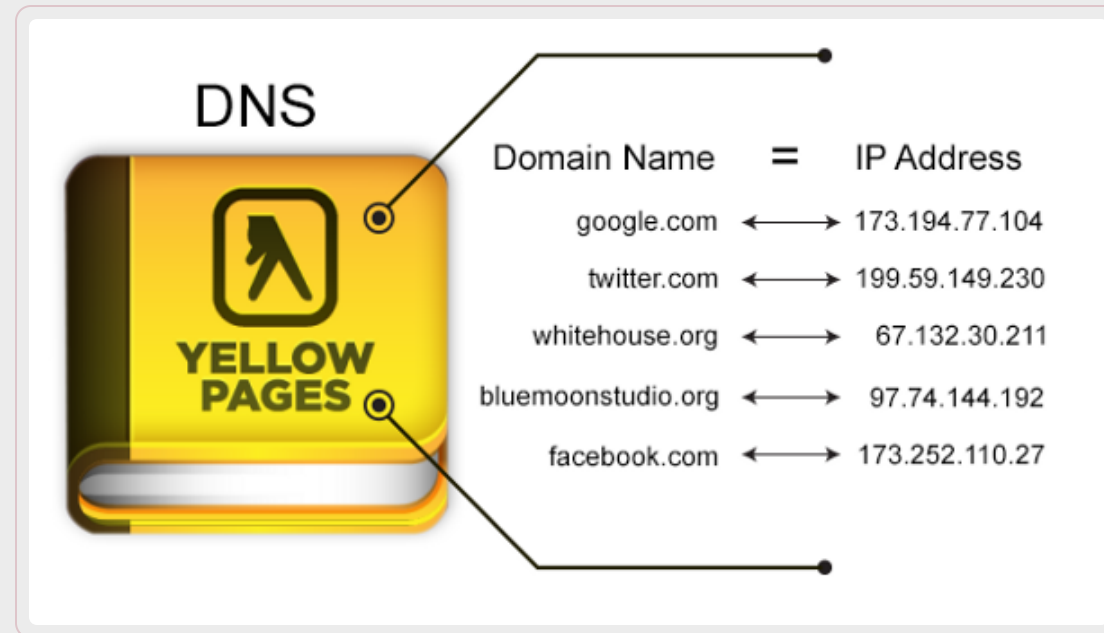
Familia de protocolos TCP/IP



Fuente: Wikipedia.org

DNS

Sistema de nomenclatura jerárquico descentralizado para dispositivos conectados a redes IP como Internet o una red privada. Su función más habitual es la asignación de nombres a direcciones IP.



Fuente: [Wikipedia.org](https://es.wikipedia.org)

La Web

Inventada por Sir Tim Berners-Lee, CERN de Ginebra (Suiza, alrededor de 1989)

Colección de documentos en HTML y transferidos a través de HTTP

Se accede a ella usando un navegador

Cada recurso en la Web se referencia mediante una URL

Lenguajes (de marcado) de la Web:

- SGML , HTML , XML , XHTML , ...

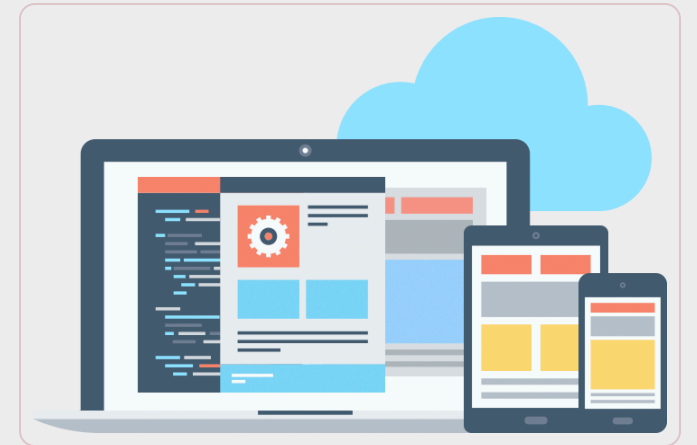


Aplicaciones web

Una aplicación web es una aplicación informática distribuida cuya interfaz de usuario es accesible desde un cliente web, normalmente un navegador web (funcionando en un PC, un teléfono móvil, una PDA, ...).

Características habituales:

- Comunicación mediante HTTP sobre TCP/IP.
- Procesamiento en servidor.
- Acceso a bases de datos.
- Arquitectura por capas.
- Distintos tipos de usuarios.



Aplicaciones web vs Sitios web

Habitualmente, se usan ambos términos de forma intercambiable

No existe una diferencia académica entre ambos términos, aunque se puede considerar que

- Una aplicación web es un tipo particular de sitio web
- En ambos casos, se trata de una forma de ofrecer/mostrar información al usuario
- En las aplicaciones web el foco está en la interacción con el usuario para llevar a cabo cierta funcionalidad



Contenidos

Introducción a las aplicaciones web

Arquitecturas web

Servicios Web

Desarrollo front-end



Arquitecturas software

La arquitectura software de una aplicación define cómo se organizan los distintos módulos que la componen.

Hay arquitecturas predefinidas, con sus ventajas e inconvenientes (patrones arquitectónicos):

- **Por Capas:** Los módulos de una capa ofrecen una funcionalidad común y solamente pueden interactuar con las adyacentes.
- **Cliente-servidor:** Aplicación distribuida que divide la funcionalidad entre un proveedor de recursos (servidor) y solicitantes de recursos (cliente).
- **Componentes:** Aplicación donde cada módulo (componente) se encarga de una funcionalidad, a la que se accede a través de un interfaz.
- **Centrada en datos:** Una base de datos es el elemento principal, a la que sólo puede acceder un módulo específico de la aplicación. El peso principal de la lógica de negocio recae en la base de datos (procedimientos almacenados)
- **Peer-to-peer (P2P):** Arquitectura distribuida que divide las tareas de forma equitativa entre distintos nodos que se comunican entre sí
- **Microservicios:** Las aplicaciones son colecciones de servicios levemente acoplados entre sí
- ...

Patrones arquitectónicos

Los patrones arquitectónicos son soluciones genéricas y reutilizables que se pueden aplicar a problemas que surgen habitualmente al diseñar arquitecturas software.

A veces se usa “patrón arquitectónico” y “estilo arquitectónico” de forma intercambiable; otros consideran los estilos como especializaciones de los patrones para un contexto concreto.

Patrones habituales en el desarrollo web (puede haber mezclas):

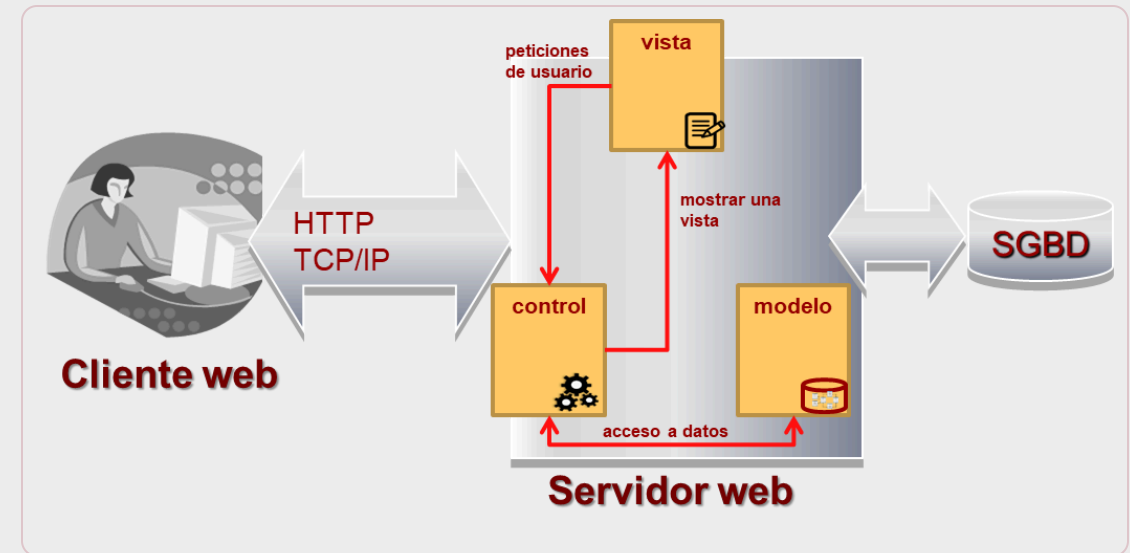
- Patrón cliente-servidor con tres capas
- Patrón modelo-vista-controlador (MVC)

Patrón: Cliente/Servidor en tres capas (Three-tiers)

La **capa de presentación** es responsable de generar la interfaz de usuario con la información proporcionada por la capa de lógica de negocio

La **capa de lógica de negocio** es responsable de implementar la lógica de la aplicación como respuesta a las peticiones de usuario, normalmente accediendo a la capa de datos.

La **capa de datos** es responsable de proporcionar acceso a los datos a la capa de lógica de negocio, normalmente accediendo a un sistema de gestión de bases de datos.



Caso específico del patrón basado en capas

Patrón: Modelo Vista Controlador (MVC)

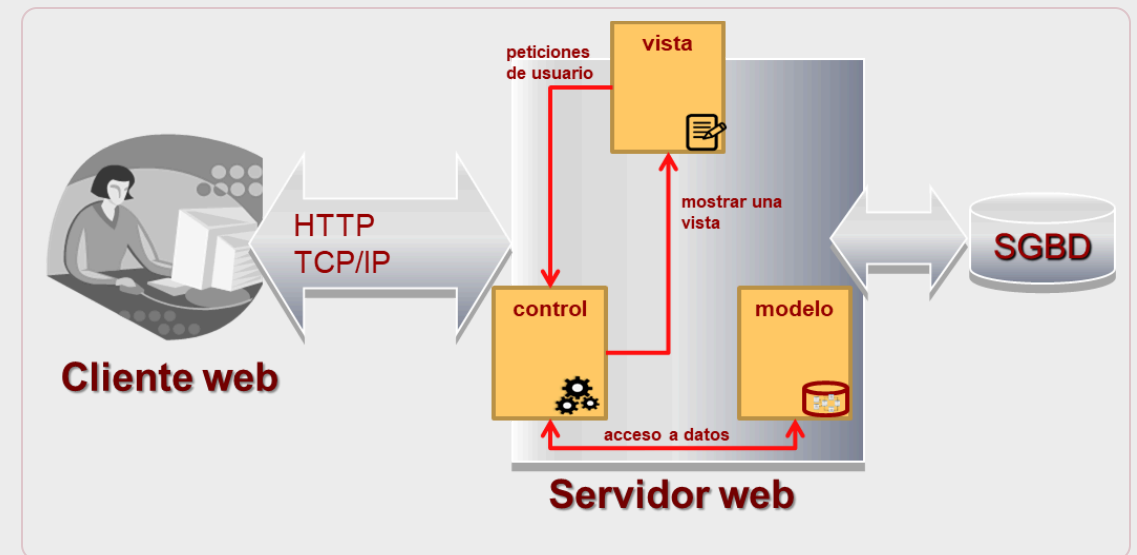
El **modelo** es responsable del acceso a datos, a través de sistema de gestión de bases de datos.

La **vista** es responsable de presentar los datos al usuario. Redirige peticiones de usuario al controlador.

El **controlador** es responsable de atender las peticiones del usuario, lo que puede implicar enviar peticiones de acceso a datos al modelo y decidir mostrar una nueva vista como respuesta.

Ejemplos de frameworks: Express, Ember, Meteor (JS), Spring (Java), Django, Flask (Python), Ruby on Rails (Ruby), Laravel (PHP)...

El más común en la actualidad. Más o menos encaja como tres capas



Caso específico del patrón basado en componentes, con tres componente

Contenidos

Introducción a las aplicaciones web

Arquitecturas web

Servicios Web

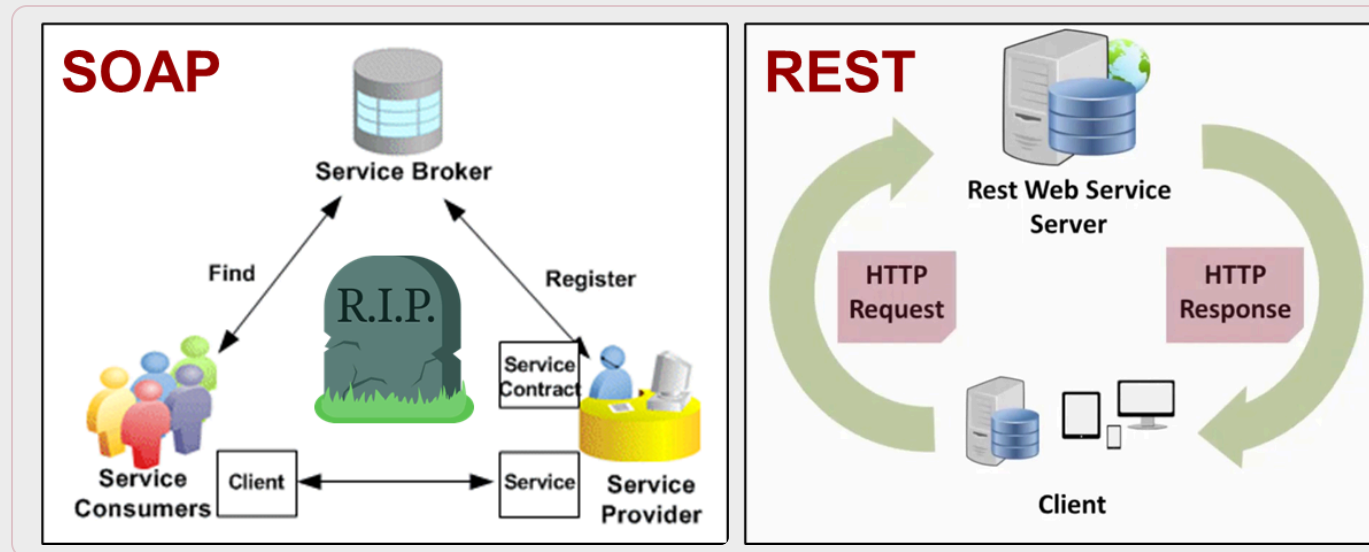
Desarrollo front-end



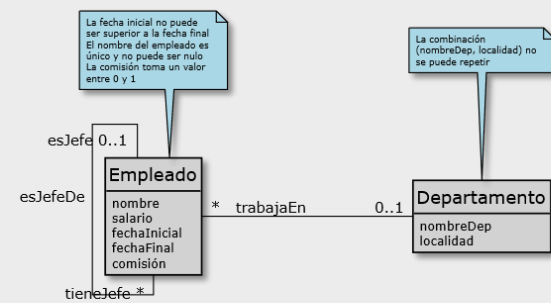
Servicios Web

Un servicio web ofrece una interfaz de programación (no de usuario) de una determinada funcionalidad (servicio) accesible a través de Internet y basada en estándares W3C.

Se transmiten datos en formato machine-readable (XML, JSON) a través de HTTP entre dos procesos



RESTful



M. Conceptual

```
CREATE TABLE Departamentos (...);
CREATE TABLE Empleados (...);
```

M. Tecnológico

-- Intensión relacional
Departamentos = { departamentold, nombreDep, localidad }
PK(departamentold)
AK(nombreDep, localidad)

Empleados = { empleadold, departamentold, jefeld, nombre, salario, fechaInicial, fechaFinal, comisión }
PK(empleadold)
FK(departamentold)/Departamentos
FK(jefeld)/Empleados

M. Relacional

Departamentos (base http://ejemplo.es/api/v1)		
Method	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos

Departamentos (base http://ejemplo.es/api/v1)		
Method	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos?sortBy={[+ -]campos}	Obtiene los departamentos ordenados por {campos} con orden creciente (+) o decreciente (-)
GET	/departamentos?fields={campos}	Obtiene los {campos} de todos los departamentos
GET	/departamentos?offset={n1}&limit={n2}	Obtiene los departamentos que están entre las posiciones {n1} y {n2}
GET	/departamentos?nombreDep={n}	Obtiene los departamentos cuyo nombre sea {n}
GET	/departamentos?localidad={loc}	Obtiene los departamentos cuya localidad sea {loc}
GET	/departamentos/{id}	Obtiene el departamento con departamentold={id}
POST	/departamentos?nombreDep={n}&localidad={loc}	Inserta un nuevo departamento con nombre {n} y localidad {loc}
DELETE	/departamentos/{id}	Elimina el departamento con departamentold={id}
PUT (*)	/departamentos/id?nombreDep={n}	Actualiza el nombre del departamento con departamentold={id} con el valor {n}
PUT (*)	/departamentos/id?localidad={loc}	Actualiza la localidad del departamento con departamentold={id} con el valor {loc}

API Rest

SOAP vs REST

SOAP:

- Protocolo de mensajería
- Especificación estándar
- Fuertemente tipado
- Ligado a XML
- Pueden ser interacciones con o sin estado
- Usa interfaces estrictamente definidas y operaciones con nombres específicos para exponer la lógica de negocio
- En desuso, absolutamente **muerto**

REST:

- Estilo arquitectónico
- Sin especificación estándar
- No tipado
- No ligado a XML, puede ser JSON u otros formatos
- Interacciones sin estado
- Usa URIs y las operaciones PUT, GET, DELETE, POST ...
- Ampliamente **usado actualmente**

Contenidos

Introducción a las aplicaciones web

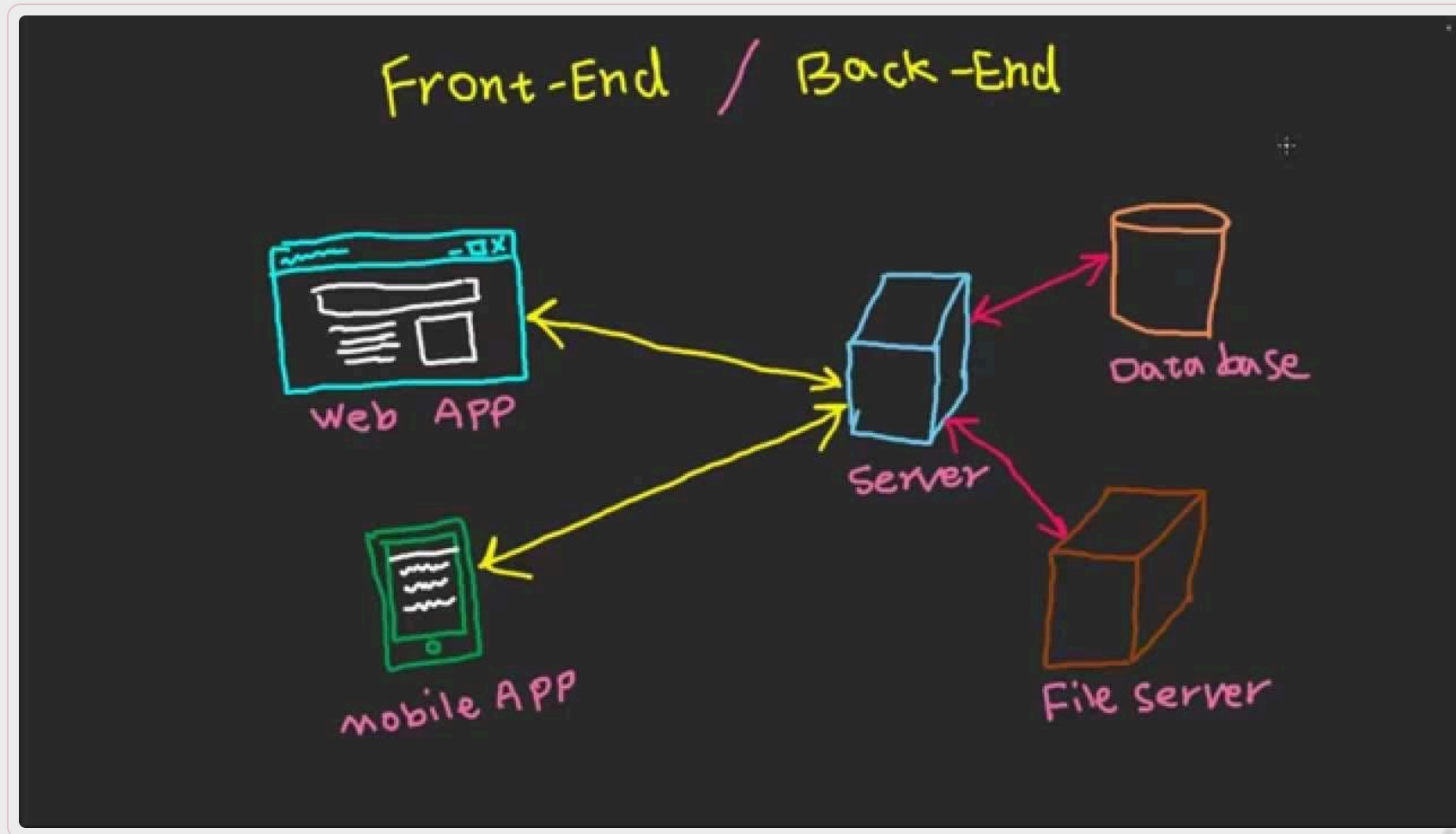
Arquitecturas web

Servicios Web

Desarrollo front-end



Front-End vs Back-End



Front-End vs Back-End

Front-End:

- Parte de la app web con la que interacciona directamente el usuario
- Implementa la estructura, diseño, comportamiento y contenido de todo lo que se puede ver en la pantalla del navegador
- Incluye: colores, estilos, imágenes, tablas, texto, botones, menús
- Importante: Responsividad y rendimiento
- Lenguajes: HTML , CSS , JavaScript
- Frameworks: Angular, React, Vue, Bootstrap...

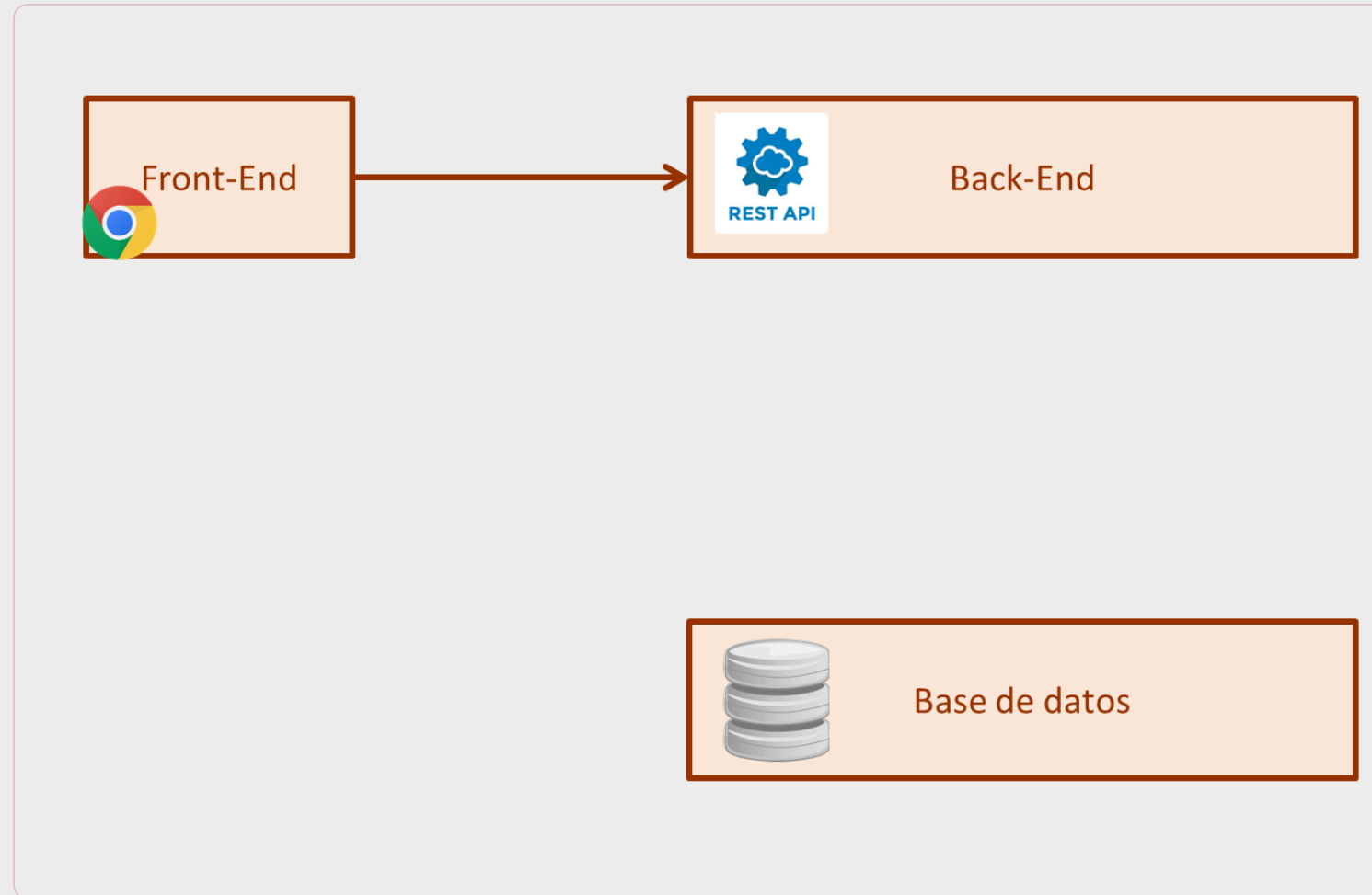
Back-End:

- Parte de la app que almacena los datos y hace que estén disponibles para el correcto funcionamiento del front-end (no hay interacción directa con el usuario).
- Su implementación incluye la definición de las APIs, lógica de negocio, acceso a la base de datos, así como otros algoritmos y procesos más complejos.
- Lenguajes: Python , Java , Ruby , JavaScript , PHP ...
- Frameworks: Django , Flask , Spring , Rails , Express , Laravel ...

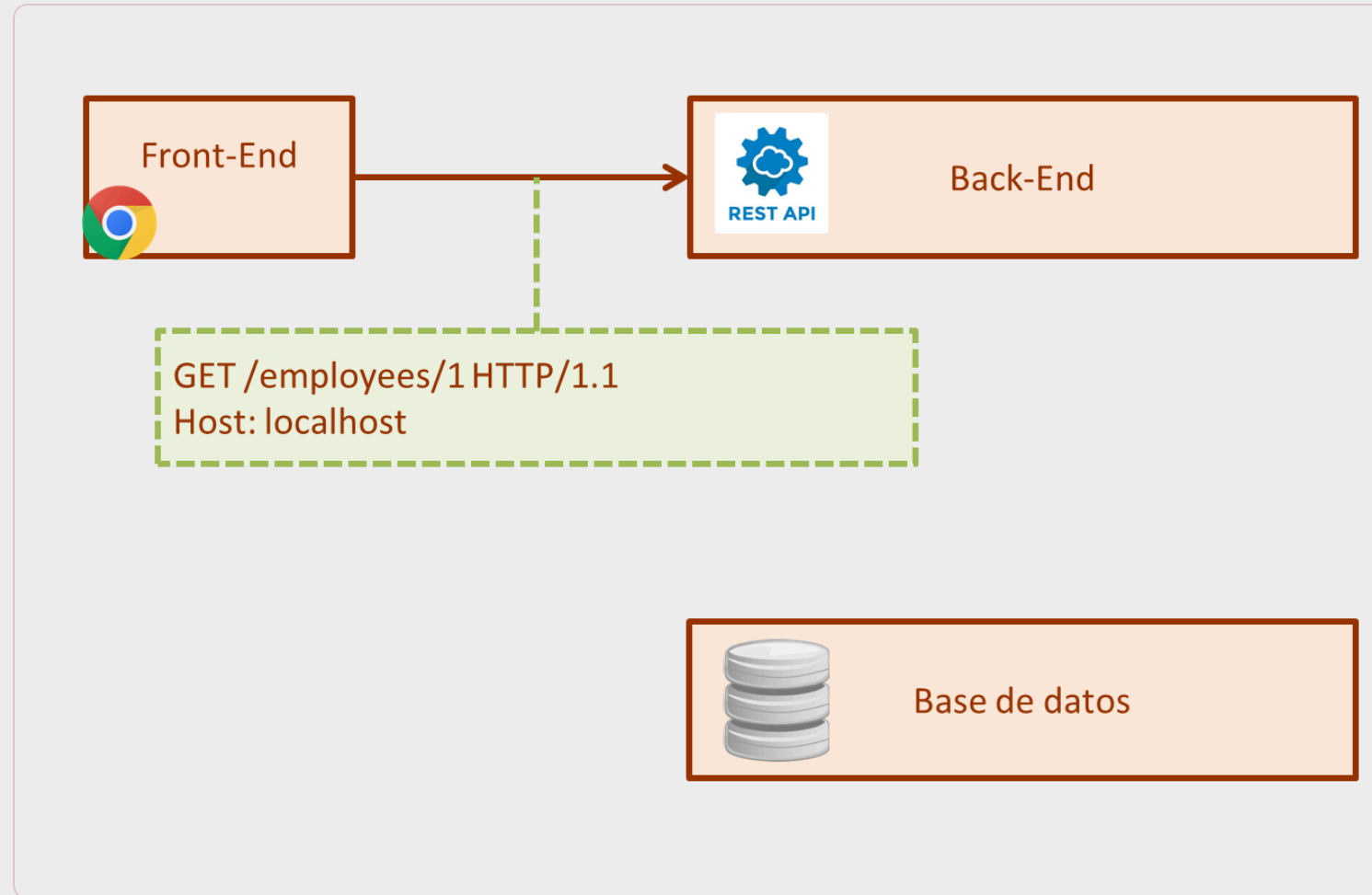
Front-End vs Back-end



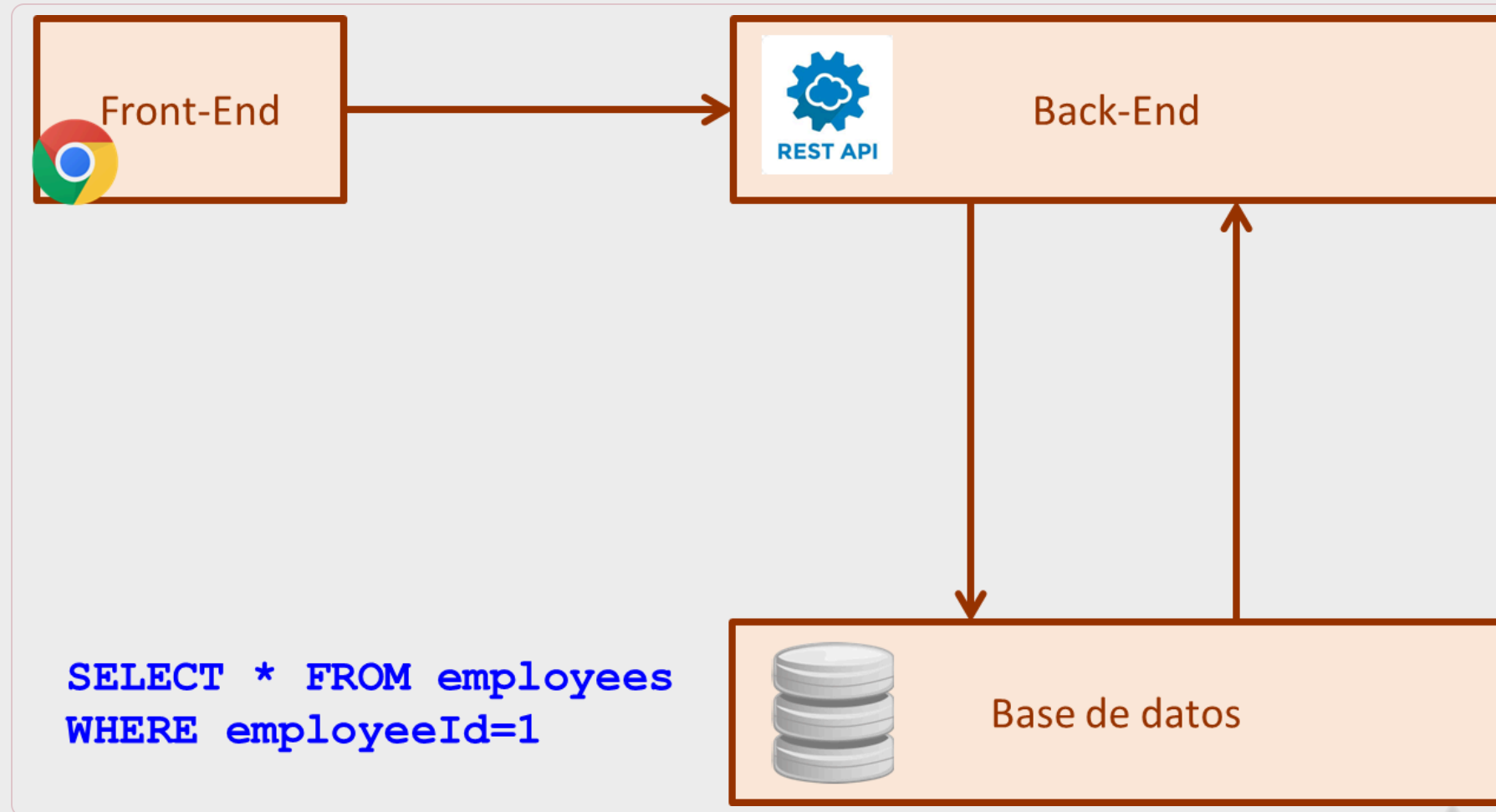
Flujo de una petición



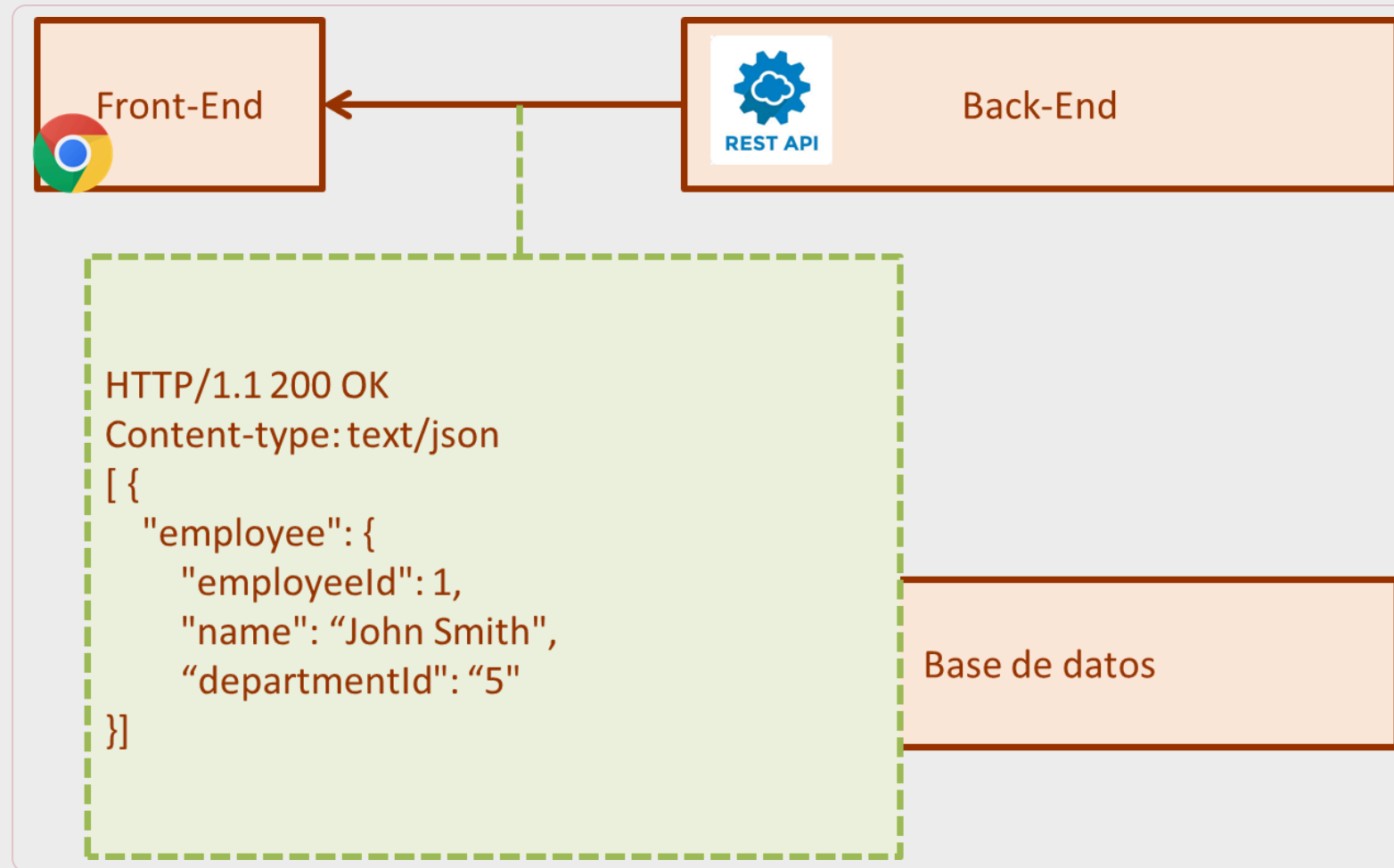
Flujo de una petición



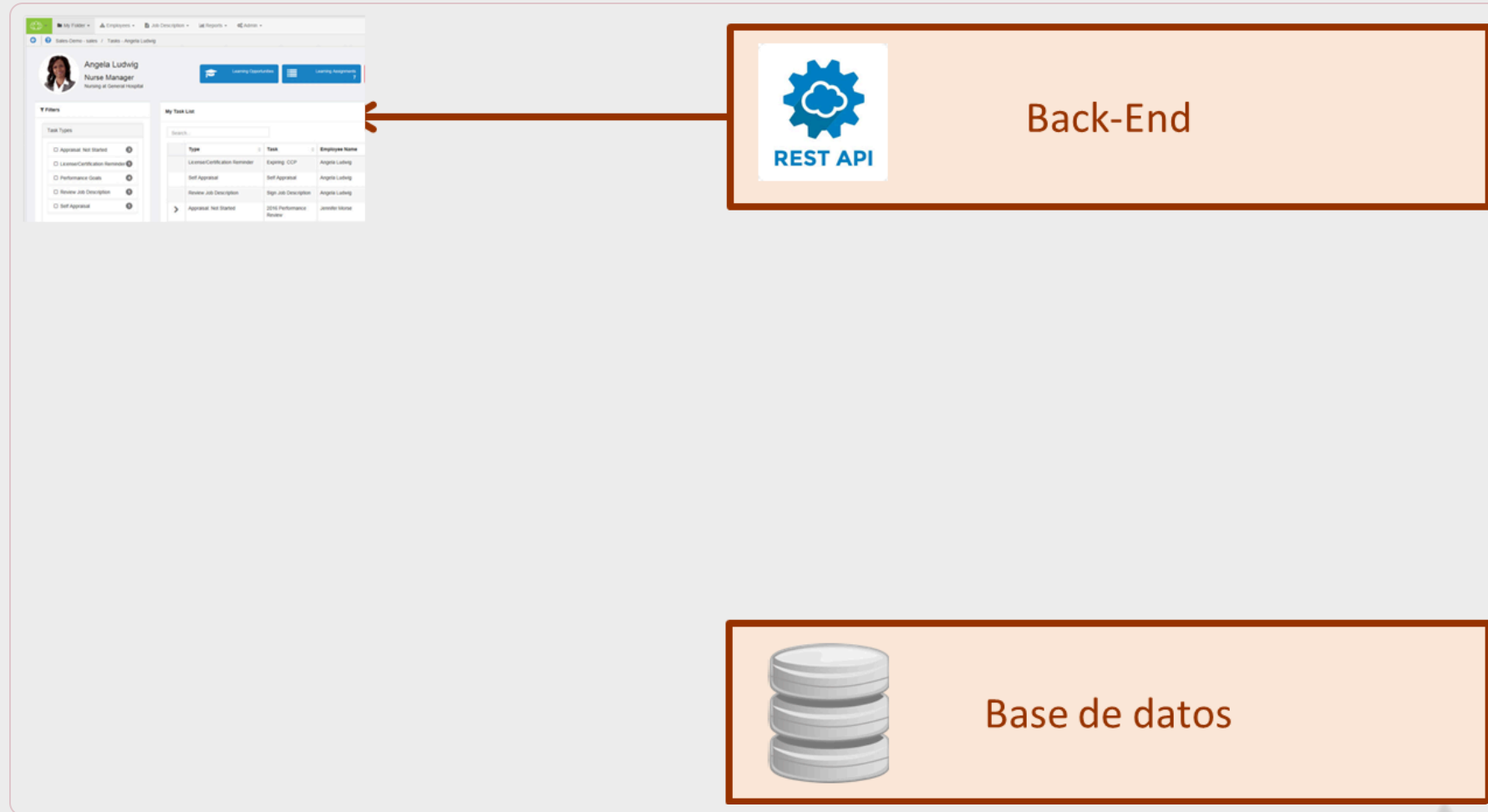
Flujo de una petición



Flujo de una petición



Flujo de una petición



Nuestro entorno



Conclusiones

Las **aplicaciones web** se construyen sobre **Internet** y la **Web**, apoyándose en protocolos como **TCP/IP**, **HTTP** y servicios como **DNS**.

La **arquitectura web** organiza la aplicación en **componentes** y **capas** para separar responsabilidades y facilitar su evolución.

Los **servicios web** permiten la **interoperabilidad** entre sistemas, siendo **REST** el estilo más habitual en aplicaciones web actuales.

El **desarrollo front-end** implementa la interfaz de usuario con **HTML**, **CSS** y **JavaScript**, coordinándose con el **back-end** mediante peticiones HTTP.



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA